UNIVERSIDAD NACIONAL DEL ALTIPLANO

Facultad de Ingenieria Mecanica, Electrica, Electronica y Sistemas

Escuela Profesional de Ingenieria de Sistemas



INFORME DE INVESTIGACIÓN

Árboles AVL

Rotaciones, balanceo e implementación

Curso: Algoritmos y Estructuras de Datos

Profesor: Mg. Aldo Hernán Zanabria Gálvez

Práctica: N. 10 (árboles AVL)

Alumno: Cristian Augusto Llanos Velasquez

Codigo: 251139

Puno – Peru – 2026

Índice

1. Actividad 1: AVL completo en C++17 **2**

1.1. Salida de la ejecución 7

1.2. Preguntas de análisis 8

2. Actividad 2 **8**

2.1. 2A: Eliminación AVL 8

2.2. 2B: BST vs. AVL en Python 8

3. Actividad 3: SIGA-UNAP-AVL **15**

4. Repositorio **15**

1 Actividad 1: AVL completo en C++17

Se implementaron `rotar_derecha`, `rotar_izquierda`, `balancear` (casos LL, RR, LR, RL) e `insertar_avl`. El factor de balance es $fb = h(der) - h(izq)$.

```cpp
#pragma once
#include <memory>
#include <algorithm>
#include <string>
#include <utility>

template<typename K, typename V = K>
struct NodoBST {
    K clave;
    V valor;
    std::unique_ptr<NodoBST> izq, der;
    int altura;
    explicit NodoBST(K k, V v=V{})
        : clave(std::move(k)), valor(std::move(v)),
          izq(nullptr), der(nullptr), altura(1) {}
};

template<typename K,typename V>
int altura(const std::unique_ptr<NodoBST<K,V>>& n) {
    return n ? n->altura : 0;
}

template<typename K,typename V>
int factor_balance(const std::unique_ptr<NodoBST<K,V>>& n) {
    return n ? altura(n->der)-altura(n->izq) : 0;
}

template<typename K,typename V>
void actualizar_altura(std::unique_ptr<NodoBST<K,V>>& n) {
    if(n) n->altura=1+std::max(altura(n->izq),altura(n->der));
}

/* TODO 1: rotar_derecha(z)
        z                 y
       / \               / \
      y   C    --->     A   z
     / \                   / \
    A   B                 B   C
*/
template<typename K,typename V>
void rotar_derecha(std::unique_ptr<NodoBST<K,V>>& z) {
    std::unique_ptr<NodoBST<K,V>> y = std::move(z->izq);
```

```
43      z->izq = std::move(y->der);    // B pasa a ser hijo izquierdo de z
44      actualizar_altura(z);          // z queda abajo: actualizar primero
45      y->der = std::move(z);         // z pasa a ser hijo derecho de y
46      z = std::move(y);              // y es la nueva raiz del subarbol
47      actualizar_altura(z);
48  }
49
50  /* TODO 2: rotar_izquierda(z) (simetrica)
51         z                 y
52        / \               / \
53       A   y     --->    z   C
54          / \           / \
55         B   C         A   B
56  */
57  template<typename K,typename V>
58  void rotar_izquierda(std::unique_ptr<NodoBST<K,V>>& z) {
59      std::unique_ptr<NodoBST<K,V>> y = std::move(z->der);
60      z->der = std::move(y->izq);
61      actualizar_altura(z);
62      y->izq = std::move(z);
63      z = std::move(y);
64      actualizar_altura(z);
65  }
66
67  // TODO 3: balancear(n) -- casos LL, RR, LR, RL
68  // fb = altura(der) - altura(izq)
69  template<typename K,typename V>
70  void balancear(std::unique_ptr<NodoBST<K,V>>& n) {
71      actualizar_altura(n);
72      int fb = factor_balance(n);
73      if (fb < -1) {
74          if (factor_balance(n->izq) > 0)   // cargado a la izquierda
75              rotar_izquierda(n->izq);
76          rotar_derecha(n);                 // caso LR
77      } else if (fb > 1) {                  // caso LL (o LR tras la
            rotacion)
78          if (factor_balance(n->der) < 0)   // cargado a la derecha
79              rotar_derecha(n->der);
80          rotar_izquierda(n);               // caso RL
81      }
82  }
83
84  // TODO 4: insertar_avl
85  template<typename K,typename V>
86  void insertar_avl(std::unique_ptr<NodoBST<K,V>>& raiz,
87                    K clave, V valor=V{}) {
88      if (!raiz) {
```

```
89          raiz = std::make_unique<NodoBST<K,V>>(std::move(clave), std::
                move(valor));
90          return;
91      }
92      if (clave < raiz->clave)
93          insertar_avl(raiz->izq, std::move(clave), std::move(valor));
94      else if (raiz->clave < clave)
95          insertar_avl(raiz->der, std::move(clave), std::move(valor));
96      else {
97          raiz->valor = std::move(valor);  // clave repetida: actualiza
                valor
98          return;
99      }
100     balancear(raiz);
101 }
102
103 // YA IMPLEMENTADA -- no modificar
104 template<typename K,typename V>
105 NodoBST<K,V>* buscar(const std::unique_ptr<NodoBST<K,V>>& raiz,
106                      const K& clave) {
107     if (!raiz) return nullptr;
108     if (clave==raiz->clave) return raiz.get();
109     if (clave<raiz->clave) return buscar(raiz->izq,clave);
110     return buscar(raiz->der,clave);
111 }
112
113 // ----------------- Actividad 2A: eliminacion -----------------
114 template<typename K,typename V>
115 NodoBST<K,V>* minimo(const std::unique_ptr<NodoBST<K,V>>& n) {
116     return n->izq ? minimo(n->izq) : n.get();
117 }
118
119 template<typename K,typename V>
120 void eliminar_avl(std::unique_ptr<NodoBST<K,V>>& raiz,
121                   const K& clave) {
122     if (!raiz) return;
123     if (clave < raiz->clave)
124         eliminar_avl(raiz->izq, clave);
125     else if (clave > raiz->clave)
126         eliminar_avl(raiz->der, clave);
127     else {
128         if (!raiz->izq && !raiz->der) {
129             raiz = nullptr;                          // Caso 1: hoja
130         } else if (!raiz->izq) {
131             raiz = std::move(raiz->der);             // Caso 2: solo hijo
                    der
132         } else if (!raiz->der) {
```

```
133             raiz = std::move(raiz->izq);           // Caso 2: solo hijo
                    izq
134         } else {                                   // Caso 3: dos hijos
135             NodoBST<K,V>* suc = minimo(raiz->der); // sucesor inorden
136             K clave_suc = suc->clave;              // copia (el nodo se
                    destruira)
137             raiz->clave = suc->clave;
138             raiz->valor = suc->valor;
139             eliminar_avl(raiz->der, clave_suc);
140         }
141     }
142     if (raiz) balancear(raiz); // reequilibrar
143 }
```

Listing 1: avl.h

```
1  #include <iostream>
2  #include <cassert>
3  #include <cstdlib>
4  #include <random>
5  #include "avl.h"
6
7  // Funcion auxiliar: retorna altura del arbol
8  template<typename K,typename V>
9  int h(const std::unique_ptr<NodoBST<K,V>>& r) { return r?r->altura:0; }
10
11 // Verifica el invariante AVL (|fb|<=1, alturas consistentes y orden
       BST)
12 template<typename K,typename V>
13 bool es_avl(const std::unique_ptr<NodoBST<K,V>>& n) {
14     if (!n) return true;
15     int hi = altura(n->izq), hd = altura(n->der);
16     if (n->altura != 1 + std::max(hi, hd)) return false;
17     if (std::abs(hd - hi) > 1) return false;
18     if (n->izq && !(n->izq->clave < n->clave)) return false;
19     if (n->der && !(n->clave < n->der->clave)) return false;
20     return es_avl(n->izq) && es_avl(n->der);
21 }
22
23 void test_P1_insercion_simple() {
24     // Insertar 3 claves ordenadas --- en BST seria h=3; en AVL h=2
25     std::unique_ptr<NodoBST<int,int>> r=nullptr;
26     insertar_avl(r,10,10); insertar_avl(r,20,20); insertar_avl(r,30,30)
           ;
27     // provoca rotacion RR (rotar_izquierda)
28     assert(h(r)==2 && "P1: altura debe ser 2 tras RR rotation");
29     assert(r->clave==20 && "P1: raiz debe ser 20");
```

```
30      std::cout << "[P1 OK] insercion_simple" << std::endl;
31  }
32
33  void test_P2_caso_LR() {
34      std::unique_ptr<NodoBST<int,int>> r=nullptr;
35      insertar_avl(r,30,30); insertar_avl(r,10,10); insertar_avl(r,20,20)
            ;
36      // provoca rotacion LR
37      assert(r->clave==20 && "P2: raiz debe ser 20");
38      assert(h(r)==2 && "P2: altura debe ser 2");
39      std::cout << "[P2 OK] caso_LR" << std::endl;
40  }
41
42  void test_P3_insercion_ordenada_grande() {
43      // Insertar 1..15 en orden --- BST degeneraria h=15
44      std::unique_ptr<NodoBST<int,int>> r=nullptr;
45      for(int i=1;i<=15;i++) insertar_avl(r,i,i);
46      assert(h(r)<=4 && "P3: altura debe ser <= 4 para 15 nodos AVL");
47      assert(buscar(r,8) && "P3: nodo 8 debe existir");
48      std::cout << "[P3 OK] insercion_ordenada_grande h=" << h(r) << std
            ::endl;
49  }
50
51  void test_P4_eliminar_hoja() {
52      std::unique_ptr<NodoBST<int,int>> r=nullptr;
53      for(int x:{20,10,30}) insertar_avl(r,x,x);
54      eliminar_avl(r,10); // hoja izquierda
55      assert(!buscar(r,10) && "P4: nodo 10 debe haberse eliminado");
56      assert(h(r)<=2 && "P4: arbol sigue balanceado");
57      std::cout << "[P4 OK] eliminar_hoja" << std::endl;
58  }
59
60  void test_P5_eliminar_raiz_dos_hijos() {
61      std::unique_ptr<NodoBST<int,int>> r=nullptr;
62      for(int x:{20,10,30,25,35}) insertar_avl(r,x,x);
63      eliminar_avl(r,20); // nodo raiz con 2 hijos
64      assert(!buscar(r,20) && "P5: nodo 20 eliminado");
65      assert(h(r)<=3 && "P5: arbol sigue balanceado");
66      std::cout << "[P5 OK] eliminar_raiz_dos_hijos" << std::endl;
67  }
68
69  void test_P6_eliminar_con_rebalanceo() {
70      // Insertar 1..7, eliminar 1 y verificar balanceo
71      std::unique_ptr<NodoBST<int,int>> r=nullptr;
72      for(int i=1;i<=7;i++) insertar_avl(r,i,i);
73      int h_antes = h(r);
74      eliminar_avl(r,1);
```

```
75      // Verificar que el arbol sigue siendo AVL
76      assert(h(r)<=h_antes && "P6: altura no aumento tras eliminacion");
77      std::cout << "[P6 OK] eliminar_con_rebalanceo" << std::endl;
78  }
79
80  // Prueba extra (no pedida): 2000 inserciones y 1000 eliminaciones
        aleatorias
81  void test_extra_invariante_aleatorio() {
82      std::mt19937 gen(42);
83      std::unique_ptr<NodoBST<int,int>> r=nullptr;
84      for (int i=0;i<2000;i++) {
85          insertar_avl(r, (int)(gen()%5000), i);
86          assert(es_avl(r) && "extra: invariante AVL tras insertar");
87      }
88      for (int i=0;i<1000;i++) {
89          eliminar_avl(r, (int)(gen()%5000));
90          assert(es_avl(r) && "extra: invariante AVL tras eliminar");
91      }
92      std::cout << "[EXTRA OK] invariante AVL en insercion/eliminacion
            aleatoria"
93                << std::endl;
94  }
95
96  int main() {
97      test_P1_insercion_simple();
98      test_P2_caso_LR();
99      test_P3_insercion_ordenada_grande();
100     std::cout << "=== Todos los casos P1-P3 pasaron ===" << std::endl;
101     test_P4_eliminar_hoja();
102     test_P5_eliminar_raiz_dos_hijos();
103     test_P6_eliminar_con_rebalanceo();
104     test_extra_invariante_aleatorio();
105     std::cout << "=== Todos los casos P4-P6 pasaron ===" << std::endl;
106     return 0;
107 }
```

Listing 2: main.cpp – pruebas

1.1 Salida de la ejecución

Compilado con `g++ -std=c++17 -Wall -Wextra`:

```
[P1 OK] insercion_simple
[P2 OK] caso_LR
[P3 OK] insercion_ordenada_grande h=4
=== Todos los casos P1-P3 pasaron ===
[P4 OK] eliminar_hoja
```

```
[P5 OK] eliminar_raiz_dos_hijos
[P6 OK] eliminar_con_rebalanceo
[EXTRA OK] invariante AVL en insercion/eliminacion aleatoria
=== Todos los casos P4-P6 pasaron ===
```

1.2 Preguntas de análisis

P1-A. Al insertar 10, 20, 30 ocurre 1 rotación, de tipo RR (simple a la izquierda sobre el 10). La raíz queda en 20 y la altura es 2. (El comentario del enunciado dice «LL», pero la secuencia crece hacia la derecha, por lo que el caso correcto es RR.)

P1-B. Al insertar 30, 10, 20 ocurren 2 rotaciones, de tipo LR (doble): primero rotar izquierda sobre 10 y luego rotar derecha sobre 30. La raíz queda en 20.

P1-C. El mínimo de nodos de un AVL de altura h cumple $N(h) = N(h-1) + N(h-2) + 1$, con $N = 1, 2, 4, 7, 12, 20, \ldots$ Como $12 \leq 15 < 20$, un AVL de 15 nodos tiene altura entre 4 y 5. P3 acepta $h \leq 4$ porque insertar 1..15 en orden produce un árbol perfecto ($2^4 - 1 = 15$), que es la altura mínima posible; es un caso favorable, no la cota general (que sería 5).

2 Actividad 2

2.1 2A: Eliminación AVL

`eliminar_avl` (incluida en `avl.h`, listado anterior) maneja los tres casos del BST (hoja, un hijo, dos hijos con sucesor inorden) y llama a `balancear` al volver de la recursión. Pasa P4–P6 y una prueba extra con 2000 inserciones y 1000 eliminaciones aleatorias verificando el invariante AVL tras cada operación.

2.2 2B: BST vs. AVL en Python

```python
import random
import math
import sys
from dataclasses import dataclass
from time import perf_counter


class Nodo:
    def __init__(self, clave, valor=None):
        self.clave = clave
        self.valor = valor
        self.izq = self.der = None
```

```
13          self.altura = 1
14
15
16  # ------------------------- Funciones AVL -------------------------
17  def height(n):
18      return n.altura if n else 0
19
20
21  def fb(n):
22      """Factor de balance = altura(der) - altura(izq)."""
23      return height(n.der) - height(n.izq) if n else 0
24
25
26  def actualizar(n):
27      n.altura = 1 + max(height(n.izq), height(n.der))
28
29
30  def rotar_der(z):
31      y = z.izq
32      z.izq = y.der
33      y.der = z
34      actualizar(z)
35      actualizar(y)
36      return y
37
38
39  def rotar_izq(z):
40      y = z.der
41      z.der = y.izq
42      y.izq = z
43      actualizar(z)
44      actualizar(y)
45      return y
46
47
48  def balancear(n):
49      actualizar(n)
50      f = fb(n)
51      if f < -1:              # cargado a la izquierda
52          if fb(n.izq) > 0:   # caso LR
53              n.izq = rotar_izq(n.izq)
54          return rotar_der(n) # caso LL
55      if f > 1:               # cargado a la derecha
56          if fb(n.der) < 0:   # caso RL
57              n.der = rotar_der(n.der)
58          return rotar_izq(n) # caso RR
59      return n
```

```
60
61
62  def insertar_avl(raiz, clave, valor=None):
63      if raiz is None:
64          return Nodo(clave, valor)
65      if clave < raiz.clave:
66          raiz.izq = insertar_avl(raiz.izq, clave, valor)
67      elif clave > raiz.clave:
68          raiz.der = insertar_avl(raiz.der, clave, valor)
69      else:
70          raiz.valor = valor  # clave repetida: actualiza el valor
71          return raiz
72      return balancear(raiz)
73
74
75  def buscar(raiz, clave):
76      while raiz is not None:
77          if clave == raiz.clave:
78              return raiz
79          raiz = raiz.izq if clave < raiz.clave else raiz.der
80      return None
81
82
83  # ------------------------- BST simple -------------------------
84  def insertar_bst(raiz, clave):  # BST simple sin balanceo
85      if raiz is None:
86          return Nodo(clave)
87      if clave < raiz.clave:
88          raiz.izq = insertar_bst(raiz.izq, clave)
89      elif clave > raiz.clave:
90          raiz.der = insertar_bst(raiz.der, clave)
91      return raiz
92
93
94  def altura_arbol(n):
95      if n is None:
96          return 0
97      return 1 + max(altura_arbol(n.izq), altura_arbol(n.der))
98
99
100 def experimento(n, orden='random'):
101     claves = list(range(1, n + 1))
102     if orden == 'random':
103         random.shuffle(claves)
104     # BST simple
105     raiz_bst = None
106     sys.setrecursionlimit(n + 100)
```

```
    for c in claves:
        raiz_bst = insertar_bst(raiz_bst, c)
    h_bst = altura_arbol(raiz_bst)
    # AVL
    raiz_avl = None
    for c in claves:
        raiz_avl = insertar_avl(raiz_avl, c)
    h_avl = raiz_avl.altura if raiz_avl else 0
    limite_teorico = int(1.44 * math.log2(n + 1))
    return h_bst, h_avl, limite_teorico


def tabla_2b():
    random.seed(2026)  # reproducible
    print(f"{'n':>8} | {'BST (ord)':>10} | {'AVL (ord)':>10} | "
          f"{'Teorico':>10} | {'BST (rnd)':>10} | {'AVL (rnd)':>10}")
    print('-' * 68)
    for n in [100, 500, 1000, 5000, 10000]:
        h_bst_o, h_avl_o, teo = experimento(n, 'ordered')
        h_bst_r, h_avl_r, _ = experimento(n, 'random')
        print(f"{n:>8} | {h_bst_o:>10} | {h_avl_o:>10} | {teo:>10} | "
              f"{h_bst_r:>10} | {h_avl_r:>10}")


# ------------------------- Actividad 3: SIGA-UNAP-AVL
    -------------------------
@dataclass
class EstudianteUNAP:
    codigo: str
    nombre: str
    escuela: str
    promedio: float


class SIGA_AVL:
    def __init__(self):
        self.raiz = None
        self.total = 0

    def matricular(self, est: EstudianteUNAP):
        t0 = perf_counter()
        existe = buscar(self.raiz, est.codigo) is not None
        self.raiz = insertar_avl(self.raiz, est.codigo, est)
        if not existe:
            self.total += 1
        t1 = perf_counter()
        accion = 'Actualizado' if existe else 'Matriculado'
```

```
153         print(f'{accion} {est.nombre} en {(t1 - t0) * 1e6:.2f} us')
154
155     def buscar_estudiante(self, codigo: str):
156         t0 = perf_counter()
157         nodo = buscar(self.raiz, codigo)
158         t1 = perf_counter()
159         if nodo:
160             print(f'Encontrado: {nodo.valor} ({(t1 - t0) * 1e6:.2f} us)
                ')
161         else:
162             print(f'Codigo {codigo} no encontrado ({(t1 - t0) * 1e6:.2f
                } us)')
163
164     def listar_ordenado(self):
165         # recorrido inorden (iterativo) => lista ordenada por codigo
166         pila, actual, i = [], self.raiz, 0
167         if actual is None:
168             print('(sin estudiantes)')
169             return
170         print(f"{'#':>3} {'Codigo':<12}{'Nombre':<16}{'Escuela':<10}{'
            Prom.':>6}")
171         while pila or actual:
172             while actual:
173                 pila.append(actual)
174                 actual = actual.izq
175             actual = pila.pop()
176             e = actual.valor
177             i += 1
178             print(f'{i:>3} {e.codigo:<12}{e.nombre:<16}{e.escuela:<10}{
                e.promedio:>6.1f}')
179             actual = actual.der
180
181     def estadisticas(self):
182         h = self.raiz.altura if self.raiz else 0
183         teo = int(1.44 * math.log2(self.total + 1)) if self.total > 0
            else 0
184         print(f'Total estudiantes: {self.total}')
185         print(f'Altura AVL actual: {h}')
186         print(f'Altura teorica max: {teo}')
187         print(f"Eficiencia: {'OPTIMO' if h <= teo else 'SUBOPTIMO'}")
188
189
190 MUESTRA = [
191     EstudianteUNAP('2026100001', 'Ana Quispe', 'Sistemas', 15.5),
192     EstudianteUNAP('2026100002', 'Luis Mamani', 'Sistemas', 14.0),
193     EstudianteUNAP('2026100003', 'Rosa Condori', 'IA&CD', 17.2),
194     EstudianteUNAP('2026100004', 'Ivan Huanca', 'Sistemas', 13.8),
```

```
    EstudianteUNAP('2026100005', 'Maria Apaza', 'IA&CD', 16.9),
    EstudianteUNAP('2026100006', 'Carlos Tito', 'Sistemas', 12.1),
    EstudianteUNAP('2026100007', 'Lucia Ramos', 'IA&CD', 18.0),
    EstudianteUNAP('2026100008', 'Pedro Vargas', 'Sistemas', 15.0),
    EstudianteUNAP('2026100009', 'Sofia Luna', 'IA&CD', 16.4),
    EstudianteUNAP('2026100010', 'Juan Puma', 'Sistemas', 14.7),
]


def menu():
    siga = SIGA_AVL()
    print('Cargando 10 estudiantes...')
    for e in MUESTRA:
        siga.matricular(e)
    while True:
        print('\n== SIGA-UNAP-AVL ==')
        print('1. Matricular nuevo estudiante')
        print('2. Buscar por codigo')
        print('3. Listar todos (inorden)')
        print('4. Estadisticas del arbol')
        print('5. Salir')
        try:
            op = input('Opcion: ').strip()
        except EOFError:
            break
        if op == '1':
            cod = input('Codigo: ').strip()
            nom = input('Nombre: ').strip()
            esc = input('Escuela: ').strip()
            try:
                prom = float(input('Promedio: '))
            except ValueError:
                print('Promedio invalido, operacion cancelada.')
                continue
            siga.matricular(EstudianteUNAP(cod, nom, esc, prom))
        elif op == '2':
            siga.buscar_estudiante(input('Codigo: ').strip())
        elif op == '3':
            siga.listar_ordenado()
        elif op == '4':
            siga.estadisticas()
        elif op == '5':
            break
        else:
            print('Opcion no valida.')


```

```
242 if __name__ == '__main__':
243     if len(sys.argv) > 1 and sys.argv[1] == '--tabla':
244         tabla_2b()  # solo la tabla de la Actividad 2B
245     else:
246         tabla_2b()
247         menu()
```

Listing 3: avl_python.py

Resultados (semilla aleatoria 2026)

Cuadro 1: Comparación de alturas BST vs. AVL

n	BST (ord)	AVL (ord)	Teórico	BST (rnd)	AVL (rnd)
100	100	7	9	12	8
500	500	9	12	18	11
1000	1000	10	14	20	12
5000	5000	13	17	27	15
10000	10000	14	19	29	16

P2B-A. La altura del BST con inserción ordenada es exactamente n: el árbol se degenera en una lista enlazada y la búsqueda cuesta $O(n)$.

P2B-B. No. $1{,}44 \log_2(10001) \approx 19{,}13$, es decir, límite 19. El AVL mide 14 (ordenado) y 16 (aleatorio): 5 y 3 niveles por debajo.

P2B-C. La premisa no es exacta: las alturas no son idénticas (14 vs. 16 con $n = 10000$), pero ambas son $O(\log n)$ y muy cercanas. El AVL mantiene $|fb| \leq 1$ en cada nodo mediante rotaciones, sin importar el orden de llegada. La inserción ordenada da un árbol casi perfecto ($\lceil \log_2(n+1) \rceil$); la aleatoria queda algo más alta porque el AVL tolera desbalances de 1.

3 Actividad 3: SIGA-UNAP-AVL

La clase `SIGA_AVL` (en `avl_python.py`) usa el AVL con el código de estudiante como clave. Ofrece matricular, buscar (mostrando μs), listar en inorden y estadísticas. Un código repetido actualiza los datos en lugar de duplicar.

Ejemplo de ejecución (carga de 10 estudiantes, luego opciones 3, 2, 1 y 4):

```
  #   Codigo       Nombre           Escuela     Prom.
  1   2026100001   Ana Quispe       Sistemas    15.5
  2   2026100002   Luis Mamani      Sistemas    14.0
...
 10   2026100010   Juan Puma        Sistemas    14.7
Total estudiantes: 10
Altura AVL actual: 4
Altura teorica max: 4
Eficiencia: OPTIMO

Encontrado: EstudianteUNAP(codigo='2026100007', nombre='Lucia Ramos',
    ...) (2.23 us)
Codigo 999 no encontrado (1.50 us)
Matriculado Leo Cruz -> Total 11, altura 4, teorica 5, OPTIMO
```

Los tiempos en μs varían en cada ejecución.

4 Repositorio

Código fuente y README en GitHub: https://github.com/Augusto-ox/Avl-Semana_10